



# Tecnológico de Monterrey

## **Mini-Triton Parser**

Desarrollo e implantación de sistemas de software

Grupo 504

Alexei Delgado De Gante | A01637405

Luis Fernando Cuevas Arroyo | A01647254

Aaron Hernandez Jimenez | A01642529

Sebastian Cervera Maltos | A01068436

Domingo 31 de mayo del 2026

## 1. Gramática (EBNF)

Se define una gramática libre de contexto en notación EBNF. Terminales entre comillas simples; no-terminales en minúsculas; ? = cero o una ocurrencia; \* = cero o más; | = alternativa; e = cadena vacía.

```

program      ::= decorator kernel EOF
decorator    ::= '@' ID '.' ID
               -- must be exactly @triton.jit
kernel       ::= 'def' ID '(' params ')' ':' '{' stmt_list '}'
params       ::= param (',' param)* | e
param        ::= ID (':' ID '.' ID)?
               -- optional annotation, e.g. BS : tl.constexpr
stmt_list    ::= stmt*
stmt         ::= assign | expr_stmt
               -- lookahead: current=ID, next='=' -> assign; else expr_stmt
assign       ::= ID '=' expr ';'
expr_stmt    ::= expr ';'
expr         ::= term (('+' | '-') term)*
term         ::= factor (('*' | '/') factor)*
factor       ::= NUMBER
               | '(' expr ')'
               | name_or_call
name_or_call ::= ID (':' ID)? '(' args ')'?
args         ::= arg (',' arg)* | e
arg          ::= expr      -- positional only; ID '=' inside args is INVALID

```

### Tokens del lexer

Los 17 tipos de token producidos por el lexer integrado en `mini_triton_parser.py`:

| Token           | Patrón                    | Ejemplo                 |
|-----------------|---------------------------|-------------------------|
| ID              | <code>[A-Za-z_]\w*</code> | x, BS, triton, tl, load |
| NUMBER          | <code>\d+(\.\d+)?</code>  | 0, 42, 3.14             |
| AT              | <code>@</code>            | @triton                 |
| DOT             | <code>.</code>            | tl.load                 |
| COLON           | <code>:</code>            | BS : tl.constexpr       |
| COMMA           | <code>,</code>            | x, y, z                 |
| SEMICOLON       | <code>;</code>            | y = x + 1;              |
| LPAREN / RPAREN | <code>( )</code>          | foo(x)                  |
| LBRACE / RBRACE | <code>{ }</code>          | { stmt }                |
| EQUALS          | <code>=</code>            | y = ...                 |

| Token        | Patrón         | Ejemplo     |
|--------------|----------------|-------------|
| PLUS / MINUS | + -            | $x + y - z$ |
| STAR / SLASH | * /            | $a * b / c$ |
| EOF          | (end of input) | —           |

## 2. Decisiones de Diseño

### 2.1 Bloques con llaves en lugar de indentación

Python real requiere tokens INDENT/DEDENT en el lexer. Mini-Triton usa { } como delimitadores de bloque, eliminando esa complejidad y manteniendo la gramática LL(1) sin ambigüedad.

### 2.2 Tokenización de nombres con punto (tl.load)

El lexer produce ID("tl") DOT ID("load") como tokens separados. `parseNameOrCall()` los reensambla mediante la regla ID ('.' ID)?, produciendo el string "tl.load" como callee del nodo Call. Evita un caso especial en el lexer.

### 2.3 Distinguir asignación vs sentencia de expresión

Lookahead de 2 tokens: si el token actual es ID y el siguiente es '=', se invoca `parseAssign()`; en cualquier otro caso, `parseExprStmt()`. El símbolo '=' solo aparece como operador de asignación a nivel de sentencia en Mini-Triton.

### 2.4 Rechazo de argumentos nombrados

`_reject_keyword_arg()` se llama antes de parsear cada argumento. Si detecta el patrón ID '=' dentro de la lista, lanza `ParseError` inmediatamente con mensaje y posición. Hace explícito que `mask=1` está fuera del alcance.

### 2.5 Control de flujo fuera de alcance

Al inicio de `parseStmt()` se verifica si el token es una palabra reservada no soportada (if, while, for, return...). Si lo es, se lanza un error descriptivo antes de continuar, evitando mensajes confusos.

## 3. Precedencia y Asociatividad

La precedencia se resuelve mediante la jerarquía de no-terminales: cada nivel agrupa más que el siguiente. La asociatividad izquierda se obtiene con iteración (while) en lugar de recursión derecha.

| Nivel       | No-terminal | Operadores         | Asociatividad |
|-------------|-------------|--------------------|---------------|
| 1 (highest) | factor      | ( ) calls literals | —             |
| 2           | term        | * /                | Left          |
| 3 (lowest)  | expr        | + -                | Left          |

### Ejemplo de derivación: $(x + 1) * 2$

```

expr
+-- term
+-- factor -> '(' expr ')'

```

```
|    +-- expr  
|    +-- term -> factor -> Name("x")  
|    +-- '+' term -> factor -> Number(1)  
+-- '*' factor -> Number(2)
```

```
AST: BinaryOp("*", BinaryOp("+", Name("x"), Number(1)), Number(2))
```

## 4. Diseño del AST

El AST se implementa con 9 nodos `@dataclass` de Python. Cada nodo representa una construcción sintáctica específica. Los nodos son inmutables por convención (no se modifican tras su construcción).

| Nodo     | Campos                                      | Descripción                                                                  |
|----------|---------------------------------------------|------------------------------------------------------------------------------|
| Program  | kernel: Kernel                              | Root node; wraps the single kernel in the file                               |
| Kernel   | name, params: List[Param], body: List[Stmt] | Function decorated with <code>@triton.jit</code>                             |
| Param    | name: str, annotation: Optional[str]        | Formal parameter; annotation e.g. <code>'tl.constexpr'</code>                |
| Assign   | target: Name, value: Expr                   | Assignment statement <code>id = expr</code> ;                                |
| ExprStmt | expr: Expr                                  | Expression used as statement <code>expr</code> ;                             |
| BinaryOp | op: str, left: Expr, right: Expr            | Binary operation <code>+</code> <code>-</code> <code>*</code> <code>/</code> |
| Call     | callee: str, args: List[Expr]               | Function call; callee may be <code>'tl.load'</code>                          |
| Name     | name: str, line: int                        | Variable or qualified name reference                                         |
| Number   | value: str, line: int                       | Integer or float literal                                                     |

### Ejemplo serializado — Kernel B completo

```

Program
├── Kernel(name="kernel_b", params=[x_ptr, y_ptr, n, BS: tl.constexpr])
│   ├── Assign
│   │   ├── Name("pid")
│   │   └── Call("tl.program_id")
│   │       └── Number(0)
│   ├── Assign
│   │   ├── Name("offs")
│   │   └── BinaryOp("+")
│   │       ├── BinaryOp("*")
│   │       │   ├── Name("pid")
│   │       │   └── Name("BS")
│   │       └── Call("tl.arange")
│   │           ├── Number(0)
│   │           └── Name("BS")
│   └── ExprStmt
│       └── Call("tl.store")
│           ├── BinaryOp("+", Name("y_ptr"), Name("offs"))
│           └── Call("tl.load")
│               └── BinaryOp("+", Name("x_ptr"), Name("offs"))

```

## 5. Casos de Prueba

Se incluyen 13 casos de prueba en el directorio tests/. Los casos válidos verifican construcciones aceptadas por la gramática; los inválidos verifican que el parser detecta y reporta errores correctamente.

### 5.1 Casos válidos (7)

| Archivo  | Código / descripción                                                                  | Razón de validez                                |
|----------|---------------------------------------------------------------------------------------|-------------------------------------------------|
| valid_01 | @triton.jit def add(x, y): { z = x + y; }                                             | Ejemplo A del enunciado — asignacion simple     |
| valid_02 | @triton.jit def one(x): { y = (x+1)*2; }                                              | Agrupacion con parentesis + precedencia * > +   |
| valid_03 | @triton.jit def f(a,b,c): { a = a + b * c; }                                          | Tres parametros; b*c se agrupa antes de sumar a |
| valid_04 | @triton.jit def g(x): { tl.load(x); }                                                 | ExprStmt con llamada calificada tl.load         |
| valid_05 | @triton.jit def h(x): { y = foo(x, 1, (2+3)); }                                       | Argumentos mixtos: Name, Number, BinaryOp       |
| valid_06 | @triton.jit def p(x, BS: tl.constexpr): { y = tl.arange(0,BS); }                      | Parametro anotado + llamada tl.arange           |
| valid_07 | @triton.jit def<br>kernel_b(x_ptr,y_ptr,n,BS:tl.constexpr){<br>pid/offs/tl.store... } | Ejemplo B completo — llamadas anidadas          |

### 5.2 Casos inválidos (6)

| Archivo    | Problema                        | Razón de invalidez                              |
|------------|---------------------------------|-------------------------------------------------|
| invalid_01 | Missing @triton.jit decorator   | parseDecorator() expects AT; gets ID("def")     |
| invalid_02 | Missing { } block delimiters    | parseKernel() expects LBRACE after ':'; gets ID |
| invalid_03 | Missing ; at end of statement   | parseAssign() expects SEMICOLON; gets RBRACE    |
| invalid_04 | Incomplete expr: y = x + ;      | parseFactor() expects operand; gets SEMICOLON   |
| invalid_05 | Keyword arg: tl.load(x, mask=1) | _reject_keyword_arg() detects ID '=' in args    |
| invalid_06 | Control flow: if (x) { y = 1; } | parseStmt() rejects unsupported keywords        |

## 6. Instrucciones de Ejecución

Requisitos: Python 3.8 o superior. Sin dependencias externas (stdlib únicamente).

### Uso básico

```
# Archivo valido -> VALIDO + AST
python mini_triton_parser.py tests/valid_07.tri

# Archivo invalido -> INVALIDO + error + posicion
python mini_triton_parser.py tests/invalid_05.tri

# Windows (glifos de arbol)
```

```
python -X utf8 mini_triton_parser.py <archivo.tri>
```

### Salida esperada — valid\_01.tri

```
VALIDO
Program
├─ Kernel(name="add", params=[x, y])
│   └─ Assign
│       ├── Name("z")
│       └─ BinaryOp("+")
│           ├── Name("x")
│           └─ Name("y")
```

### Salida esperada — invalid\_05.tri

```
INVALIDO: argumento con nombre 'mask=...' no permitido en línea 2, col 36
```